



RutaBusQuilla

“ingeniería de Software II”

Documentación Completa del Sistema

Análisis de Requerimientos · Diseño Arquitectónico
Diseño Detallado · Diseño de Interfaz

Integrantes:

Daniel Caballero Olivero
Mateo Cantillo Cortina
Fabio Quintero Salgado
Juan Jaimes

Docente:

Patty Pedroza

Sede Centro — Barranquilla, 2026



1. Análisis de Requerimientos

1.1 Actores del Sistema

El sistema RutaBusQuilla cuenta con dos tipos de actores principales:

Actor	Descripción	Acceso
Usuario Registrado	Ciudadano de Barranquilla que utiliza la aplicación para consultar rutas, reportar incidencias y dejar reseñas.	Funcionalidades completas de consulta e interacción
Administrador	Responsable de gestionar la información del sistema: rutas, paradas, usuarios y reportes de incidencias.	Panel de administración con control total del sistema

1.2 Requerimientos Funcionales

Los siguientes requerimientos describen las funcionalidades que el sistema debe proveer:

ID	Nombre	Descripción	Actor
RF-01	Registro de Usuario	El sistema debe permitir al usuario crear una cuenta con nombre, correo electrónico y contraseña.	Usuario
RF-02	Inicio de Sesión	El sistema debe autenticar al usuario mediante correo y contraseña, generando un token de sesión.	Usuario / Admin
RF-03	Visualizar Rutas en Mapa	El sistema debe mostrar las rutas de buses urbanos de Barranquilla sobre un mapa interactivo usando OpenStreetMap.	Usuario
RF-04	Buscar Paradas Cercanas	El sistema debe detectar la ubicación del usuario y listar las paradas de bus más cercanas en un radio configurable.	Usuario
RF-05	Consultar Detalle de Ruta	El sistema debe mostrar información detallada de una ruta: nombre, paradas, recorrido y estado actual.	Usuario
RF-06	Reportar Obstrucción	El sistema debe permitir al usuario registrado reportar una incidencia o bloqueo en una ruta específica.	Usuario
RF-07	Recibir Alertas en Tiempo Real	El sistema debe notificar a los usuarios suscritos cuando se reporte una obstrucción en una ruta que siguen.	Usuario



ID	Nombre	Descripción	Actor
RF-08	Dejar Reseña	El sistema debe permitir al usuario registrado calificar una ruta y dejar un comentario sobre la experiencia.	Usuario
RF-09	Gestionar Rutas	El administrador debe poder crear, editar y eliminar rutas y paradas desde el panel de administración.	Administrador
RF-10	Gestionar Reportes	El administrador debe poder visualizar, validar y cerrar los reportes de obstrucción enviados por usuarios.	Administrador
RF-11	Calcular Ruta a Pie	El sistema debe calcular y mostrar la ruta óptima a pie desde la ubicación del usuario hasta una parada, usando OSRM.	Usuario
RF-12	Cerrar Sesión	El sistema debe permitir al usuario y administrador cerrar su sesión activa de forma segura.	Usuario / Admin

1.3 Requerimientos No Funcionales

ID	Categoría	Descripción
RNF-01	Rendimiento	El sistema debe responder las consultas de rutas en menos de 2 segundos bajo condiciones normales de uso.
RNF-02	Disponibilidad	La aplicación debe estar disponible al menos el 95% del tiempo en horario de 5:00 AM a 11:00 PM.
RNF-03	Seguridad	Las contraseñas deben almacenarse cifradas con BCrypt. Las sesiones deben manejarse con JWT.
RNF-04	Usabilidad	La interfaz debe ser intuitiva y accesible desde navegadores modernos en dispositivos móviles y de escritorio.
RNF-05	Escalabilidad	La arquitectura en capas debe permitir agregar nuevas rutas, paradas o módulos sin reestructurar el sistema.
RNF-06	Portabilidad	El backend debe ejecutarse en contenedores Docker para facilitar el despliegue en cualquier entorno.
RNF-07	Mantenibilidad	El código debe seguir la arquitectura Controller → Service → Repository → Model para facilitar el mantenimiento.



1.4 Casos de Uso

Diagrama de Casos de Uso (descripción textual)

El sistema RutaBusQuilla contempla los siguientes casos de uso principales agrupados por actor:

Caso de Uso	Actor	Precondición	Flujo Principal	Postcondición
CU-01: Registrarse	Usuario	El usuario no tiene cuenta.	1. Ingresar datos. 2. Sistema valida. 3. Se crea la cuenta.	Usuario registrado en el sistema.
CU-02: Iniciar Sesión	Usuario / Admin	El usuario tiene cuenta activa.	1. Ingresar credenciales. 2. Sistema autentica. 3. Se genera sesión.	Sesión activa y acceso al sistema.
CU-03: Ver Rutas en Mapa	Usuario	Sesión activa.	1. Accede al mapa. 2. Sistema carga rutas. 3. Usuario visualiza recorridos.	Rutas desplegadas sobre el mapa.
CU-04: Buscar Parada Cercana	Usuario	GPS del dispositivo activo.	1. Sistema detecta ubicación. 2. Consulta paradas cercanas. 3. Lista resultados.	Lista de paradas ordenada por distancia.
CU-05: Reportar Obstrucción	Usuario	Sesión activa.	1. Selecciona ruta. 2. Describe la incidencia. 3. Sistema registra el reporte.	Reporte creado y notificación enviada.
CU-06: Gestionar Rutas	Administrador	Sesión admin activa.	1. Accede al panel. 2. Crea/edita/elimina ruta. 3. Sistema actualiza BD.	Datos de ruta actualizados.



2. Diseño Arquitectónico

2.1 Descripción General de la Arquitectura

RutaBusQuilla implementa una arquitectura en capas (Layered Architecture) dividida en dos grandes componentes: el Frontend (cliente web) y el Backend (servidor API REST). La comunicación entre ambos se realiza mediante peticiones HTTP usando el formato JSON.

2.2 Capas del Sistema

Capa	Tecnología	Responsabilidad
Presentación (Frontend)	React + Vite + React Leaflet	Renderiza la interfaz de usuario. Consume la API REST del backend y muestra mapas interactivos.
Controlador (Controller)	Spring Boot 3.5 - @RestController	Expone los endpoints REST. Recibe peticiones HTTP, valida entradas y delega al servicio correspondiente.
Servicio (Service)	Spring Boot 3.5 - @Service	Contiene la lógica de negocio: cálculo de rutas, manejo de alertas, validaciones y coordinación de repositorios.
Repositorio (Repository)	Spring Data JPA - @Repository	Define las operaciones de acceso a datos. Usa interfaces que extienden JpaRepository para consultas CRUD.
Modelo (Model)	Hibernate + JPA - @Entity	Representa las entidades del dominio mapeadas a tablas de la base de datos MySQL.
Base de Datos	MySQL (Docker)	Almacena de forma persistente usuarios, rutas, paradas, reseñas y reportes de incidencias.

2.3 Flujo de Comunicación

El flujo general de una petición en el sistema sigue el siguiente recorrido:

- El Usuario interactúa con la interfaz React (Frontend).
- El Frontend realiza una solicitud HTTP (GET/POST/PUT/DELETE) al Backend mediante Axios.
- El Controller de Spring Boot recibe la solicitud y la valida.
- El Controller invoca al Service correspondiente con los datos necesarios.
- El Service ejecuta la lógica de negocio y llama al Repository.
- El Repository consulta o persiste datos en la base de datos MySQL via Hibernate.



- La respuesta regresa por el mismo camino hasta el Frontend en formato JSON.

2.4 Diagrama de Componentes

Componente	Tipo	Puerto	Descripción
rutabusquilla-frontend	Contenedor / App Web	5173	Aplicación React servida por Vite. Accede al backend y a OSRM/OpenStreetMap.
rutabusquilla-backend	API REST Spring Boot	8080	Expone todos los endpoints del sistema. Conecta con la BD y servicios externos.
rutabusquilla-db	MySQL en Docker	3306	Base de datos relacional con esquema de rutas, usuarios, paradas y reportes.
OSRM (externo)	Servicio HTTP externo	443	Calcula rutas a pie entre dos puntos geográficos. Usado por el servicio de navegación.
OpenStreetMap (externo)	Tiles de mapa	443	Provee las imágenes del mapa interactivo renderizadas en React Leaflet.

2.5 Patrones de Diseño

Patrón	Ubicación en el Sistema	Propósito
Singleton	Configuración global de la aplicación (ApplicationContext de Spring)	Garantiza una única instancia del contexto de la aplicación, centralizando el estado global.
Strategy	Servicio de búsqueda de rutas (RutaSearchService)	Permite intercambiar el algoritmo de búsqueda de rutas (por distancia, por tiempo, por transbordos) en tiempo de ejecución.
Observer	Servicio de notificaciones (AlertaService)	Cuando un usuario reporta una obstrucción, el sistema notifica automáticamente a todos los usuarios suscritos a esa ruta.



3. Diseño Detallado

3.1 Modelo de Datos (Entidad-Relación)

El sistema utiliza MySQL como gestor de base de datos. A continuación se describen las entidades principales y sus atributos:

Entidad: Usuario

Atributo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único del usuario.
nombre	VARCHAR(100)	NOT NULL	Nombre completo del usuario.
correo	VARCHAR(150)	NOT NULL, UNIQUE	Correo electrónico usado para autenticación.
contrasena	VARCHAR(255)	NOT NULL	Contraseña cifrada con BCrypt.
rol	ENUM	NOT NULL, DEFAULT 'USER'	Rol del usuario: USER o ADMIN.
fechaRegistro	DATETIME	NOT NULL	Fecha y hora de creación de la cuenta.

Entidad: Ruta

Atributo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único de la ruta.
nombre	VARCHAR(100)	NOT NULL	Nombre o código de la ruta (ej: Ruta 1, Circular).
descripcion	TEXT	NULL	Descripción del recorrido de la ruta.
estado	ENUM	NOT NULL, DEFAULT 'ACTIVA'	Estado de la ruta: ACTIVA, SUSPENDIDA, EN_MANTENIMIENTO.

Entidad: Parada

Atributo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único de la parada.



Atributo	Tipo	Restricciones	Descripción
nombre	VARCHAR(150)	NOT NULL	Nombre o referencia de la parada.
latitud	DECIMAL(10,7)	NOT NULL	Coordenada geográfica latitud.
longitud	DECIMAL(10,7)	NOT NULL	Coordenada geográfica longitud.
ruta_id	BIGINT	FK → Ruta(id)	Ruta a la que pertenece la parada.
orden	INT	NOT NULL	Posición de la parada dentro del recorrido de la ruta.

Entidad: Reporte

Atributo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único del reporte.
descripcion	TEXT	NOT NULL	Descripción de la obstrucción o incidencia.
fechaReporte	DATETIME	NOT NULL	Fecha y hora en que se generó el reporte.
estado	ENUM	NOT NULL, DEFAULT 'ABIERTO'	Estado del reporte: ABIERTO, EN_REVISION, CERRADO.
usuario_id	BIGINT	FK → Usuario(id)	Usuario que creó el reporte.
ruta_id	BIGINT	FK → Ruta(id)	Ruta afectada por la obstrucción.

Entidad: Reseña

Atributo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único de la reseña.
calificacion	INT	NOT NULL, CHECK (1-5)	Calificación del 1 al 5 dada por el usuario.
comentario	TEXT	NULL	Comentario opcional del usuario sobre la ruta.
fecha	DATETIME	NOT NULL	Fecha en que se publicó la reseña.
usuario_id	BIGINT	FK → Usuario(id)	Usuario que dejó la reseña.
ruta_id	BIGINT	FK → Ruta(id)	Ruta que está siendo calificada.



3.2 Diagrama de Clases (descripción)

Las clases principales del backend de RutaBusQuilla y sus relaciones son:

Clase	Capa	Atributos Principales	Relaciones
Usuario	Model (@Entity)	id, nombre, correo, contraseña, rol	1 Usuario → N Reportes, N Reseñas
Ruta	Model (@Entity)	id, nombre, descripción, estado	1 Ruta → N Paradas, N Reportes, N Reseñas
Parada	Model (@Entity)	id, nombre, latitud, longitud, orden, ruta	N Paradas → 1 Ruta
Reporte	Model (@Entity)	id, descripción, fecha, estado, usuario, ruta	N Reportes → 1 Ruta, N Reportes → 1 Usuario
Resena	Model (@Entity)	id, calificación, comentario, fecha, usuario, ruta	N Reseñas → 1 Ruta, N Reseñas → 1 Usuario
UsuarioService	Service (@Service)	—	Usa UsuarioRepository. Implementa lógica de auth.
RutaService	Service (@Service)	—	Usa RutaRepository, ParadaRepository. Implementa Strategy.
AlertaService	Service (@Service)	observers: List<Usuario>	Implementa Observer para notificaciones.
UsuarioController	Controller (@RestController)	—	Endpoints: POST /auth/registro, POST /auth/login
RutaController	Controller (@RestController)	—	Endpoints: GET /rutas, GET /rutas/{id}, POST /rutas
ReporteController	Controller (@RestController)	—	Endpoints: POST /reportes, GET /reportes, PATCH /reportes/{id}



3.3 Diagrama de Secuencia: Reportar Obstrucción

A continuación se describe el flujo de mensajes entre componentes cuando un usuario registrado reporta una obstrucción en una ruta:

Paso	Origen	Destino	Mensaje / Acción
1	Usuario (Frontend)	React UI	Hace clic en 'Reportar Obstrucción' y completa el formulario.
2	React UI	Backend (ReporteController)	POST /reportes { rutald, descripcion } + token JWT en header.
3	ReporteController	ReporteService	crearReporte(reporteDTO, usuariold)
4	ReporteService	ReporteRepository	save(nuevoReporte) → persiste en MySQL.
5	ReporteService	AlertaService	notificarObstruccion(ruta, reporte)
6	AlertaService	Usuarios suscritos	Itera los observers de la ruta y envía notificación a cada uno.
7	ReporteController	React UI	HTTP 201 Created { reporteld, mensaje: 'Reporte creado exitosamente' }
8	React UI	Usuario	Muestra mensaje de confirmación en la pantalla.

4. Diseño de Interfaz

4.1 Principios de Diseño

- Simplicidad: Interfaz limpia enfocada en la tarea principal: consultar rutas.
- Responsive: Adaptada para dispositivos móviles y de escritorio.
- Feedback visual: El usuario siempre recibe confirmación de sus acciones.
- Accesibilidad: Colores contrastantes y textos legibles.



4.2 Pantallas Principales

Pantalla	Ruta URL	Descripción	Componentes Clave
Login / Registro	/login, /registro	Formulario de autenticación y creación de cuenta.	Inputs de email/contraseña, botón de acción, enlace de cambio entre login y registro.
Mapa Principal	/mapa	Vista central de la app. Muestra el mapa interactivo de Barranquilla con las rutas.	Mapa Leaflet, lista lateral de rutas, botón de geolocalización, marcadores de paradas.
Detalle de Ruta	/rutas/:id	Información completa de una ruta seleccionada.	Mapa con recorrido resaltado, lista de paradas, calificación promedio, reportes activos.
Reportar Obstrucción	/reportar	Formulario para reportar una incidencia en una ruta.	Selector de ruta, campo de descripción, botón de envío, confirmación.
Mis Reseñas	/resenas	Listado de reseñas dejadas por el usuario autenticado.	Tarjetas de reseña con calificación por estrellas y comentario.
Panel Admin - Rutas	/admin/rutas	Gestión CRUD de rutas por parte del administrador.	Tabla de rutas, botones de edición/eliminación, formulario de creación.
Panel Admin - Reportes	/admin/reportes	Visualización y gestión de los reportes de incidencias.	Tabla de reportes con estado, botones de cambio de estado (Abrir/Cerrar).

4.3 Flujo de Navegación

El flujo principal de navegación de un usuario registrado en RutaBusQuilla es el siguiente:

- Pantalla de Login → (credenciales correctas) → Mapa Principal.
- Mapa Principal → (clic en ruta) → Detalle de Ruta.
- Detalle de Ruta → (clic en 'Reportar') → Formulario de Reporte → Confirmación → Mapa Principal.
- Detalle de Ruta → (clic en 'Dejar reseña') → Formulario de Reseña → Detalle de Ruta.
- Menú lateral → Mis Reseñas.
- (Admin) Menú lateral → Panel Admin → Rutas / Reportes.



4.4 Estructura de Carpetas del Frontend

La aplicación React está organizada de la siguiente manera:

Carpeta / Archivo	Descripción
src/components/	Componentes reutilizables: MapaComponent, NavBar, RutaCard, ReporteForm, Estrellas.
src/pages/	Páginas/vistas principales: LoginPage, MapaPage, RutaDetallePage, AdminPage.
src/services/	Funciones Axios para llamar a la API: rutaService.js, usuarioService.js, reporteService.js.
src/context/	Contexto global de autenticación con React Context API (AuthContext.jsx).
src/hooks/	Hooks personalizados: useGeolocalizacion.js, useRutas.js.
src/assets/	Imágenes, íconos y estilos CSS globales.
main.jsx	Punto de entrada de la aplicación React.
vite.config.js	Configuración de Vite: proxy al backend en puerto 8080.

4.5 Estructura de Carpetas del Backend

El proyecto Spring Boot está organizado siguiendo la arquitectura en capas:

Paquete / Carpeta	Descripción
com.rutabusquilla.controller/	Clases @RestController: UsuarioController, RutaController, ReporteController, ResenaController.
com.rutabusquilla.service/	Clases @Service con lógica de negocio: UsuarioService, RutaService, AlertaService, ResenaService.
com.rutabusquilla.repository/	Interfaces @Repository que extienden JpaRepository: UsuarioRepository, RutaRepository, etc.
com.rutabusquilla.model/	Entidades JPA @Entity: Usuario, Ruta, Parada, Reporte, Resena.
com.rutabusquilla.dto/	Objetos de transferencia de datos (DTO) para request/response de la API.
com.rutabusquilla.config/	Configuración de seguridad (Spring Security + JWT) y CORS.
src/main/resources/application.properties	Configuración de conexión a MySQL, puerto del servidor, Hibernate DDL.



Paquete / Carpeta	Descripción
docker-compose.yml	Definición del contenedor MySQL: imagen, puerto, variables de entorno, volúmenes.



5. Conclusión

RutaBusQuilla es una solución de software completa que aborda la necesidad real de centralizar la información del transporte público urbano en Barranquilla. A través de su arquitectura en capas, el uso de patrones de diseño bien definidos (Singleton, Strategy, Observer) y la integración de tecnologías modernas como Spring Boot, React, React Leaflet y OSRM, el sistema logra ofrecer una experiencia de usuario fluida e intuitiva.

El proyecto demuestra la aplicación práctica de los conocimientos adquiridos en Ingeniería de Software, cubriendo el ciclo completo de desarrollo: análisis de requerimientos, diseño arquitectónico y detallado, diseño de interfaz e implementación. Las decisiones tecnológicas tomadas garantizan escalabilidad, mantenibilidad y portabilidad del sistema a futuro.